

tf.compat.v1.DeviceSpec Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/DeviceSpec

Represents a (possibly partial) specification for a TensorFlow device.

Function Signatures

```
tf.compat.v1.DeviceSpec( job=None, replica=None, task=None,
device_type=None, device_index=None )

# Place the operations on device "GPU:0" in the "ps" job.
device_spec = DeviceSpec(job="ps", device_type="GPU",
device_index=0) with tf.device(device_spec.to_string()): #
Both my_var and squared_var will be placed on
/job:ps/device:GPU:0. my_var = tf.Variable(...,
name="my_variable") squared_var = tf.square(my_var)
```

Code Examples

```
tf.compat.v1.DeviceSpec(
job=None, replica=None, task=None, device_type=None,
device_index=None
)
```

```
tf.compat.v1.DeviceSpec(  
    job=None, replica=None, task=None, device_type=None,  
    device_index=None  
)
```

```
# Place the operations on device "GPU:0" in the "ps" job.  
device_spec = DeviceSpec(job="ps", device_type="GPU",  
    device_index=0)  
with tf.device(device_spec.to_string()):  
    # Both my_var and squared_var will be placed on  
    /job:ps/device:GPU:0.  
    my_var = tf.Variable(..., name="my_variable")  
    squared_var = tf.square(my_var)
```

```
# Place the operations on device "GPU:0" in the "ps" job.  
device_spec = DeviceSpec(job="ps", device_type="GPU",  
    device_index=0)  
with tf.device(device_spec.to_string()):  
    # Both my_var and squared_var will be placed on  
    /job:ps/device:GPU:0.  
    my_var = tf.Variable(..., name="my_variable")  
    squared_var = tf.square(my_var)
```

```
tf.compat.v1.disable_eager_execution()  
# Same as previous  
device_spec = DeviceSpec(job="ps", device_type="GPU",  
    device_index=0)  
# No need of .to_string() method.  
with tf.device(device_spec):  
    my_var = tf.Variable(..., name="my_variable")  
    squared_var = tf.square(my_var)
```

```
tf.compat.v1.disable_eager_execution()
# Same as previous
device_spec = DeviceSpec(job="ps", device_type="GPU",
device_index=0)
# No need of .to_string() method.
with tf.device(device_spec):
my_var = tf.Variable(..., name="my_variable")
squared_var = tf.square(my_var)
```

```
gpu0_spec = DeviceSpec(job="ps", device_type="GPU", device_index=0)
with tf.device(DeviceSpec(job="train").to_string()):
with tf.device(gpu0_spec.to_string()):
# Nodes created here will be assigned to /job:ps/device:GPU:0.
with tf.device(DeviceSpec(device_type="GPU",
device_index=1).to_string()):
# Nodes created here will be assigned to /job:train/device:GPU:1.
```

```
gpu0_spec = DeviceSpec(job="ps", device_type="GPU", device_index=0)
with tf.device(DeviceSpec(job="train").to_string()):
with tf.device(gpu0_spec.to_string()):
# Nodes created here will be assigned to /job:ps/device:GPU:0.
with tf.device(DeviceSpec(device_type="GPU",
device_index=1).to_string()):
# Nodes created here will be assigned to /job:train/device:GPU:1.
```

Documentation

Represents a (possibly partial) specification for a TensorFlow device.

Inherits From: DeviceSpec

DeviceSpecs are used throughout TensorFlow to describe where state is stored and computations occur. Using DeviceSpec allows you to parse device spec

strings to verify their validity, merge them or compose them programmatically.

Example:

With eager execution disabled (by default in TensorFlow 1.x and by calling `disable_eager_execution()` in TensorFlow 2.x), the following syntax can be used:

If a `DeviceSpec` is partially specified, it will be merged with other `DeviceSpec`s according to the scope in which it is defined. `DeviceSpec` components defined in inner scopes take precedence over those defined in outer scopes.

A `DeviceSpec` consists of 5 components -- each of which is optionally specified:

- Job: The job name.
- Replica: The replica index.
- Task: The task index.
- Device type: The device type string (e.g. "CPU" or "GPU").
- Device index: The device index.

Attributes

Methods

from_string

[View source](#)

Construct a `DeviceSpec` from a string.

make_merged_spec

[View source](#)

Returns a new DeviceSpec which incorporates dev.

When combining specs, dev will take precedence over the current spec.

So for instance:

is equivalent to:

merge_from

[View source](#)

Merge the properties of "dev" into this DeviceSpec.

parse_from_string

[View source](#)

Parse a DeviceSpec name into its components.

2.x behavior change:

In TensorFlow 1.x, this function mutates its own state and returns itself.

In 2.x, DeviceSpecs are immutable, and this function will return a DeviceSpec which contains the spec.

- Recommended:

```
# my_spec and my_updated_spec are unrelated.
```

```
my_spec = tf.DeviceSpec.from_string("/CPU:0")
```

```
my_updated_spec = tf.DeviceSpec.from_string("/GPU:0")
```

```
with tf.device(my_updated_spec):
```

```
...
```

- Will work in 1.x and 2.x (though deprecated in 2.x):

```
my_spec = tf.DeviceSpec.from_string("/CPU:0")
my_updated_spec = my_spec.parse_from_string("/GPU:0")
with tf.device(my_updated_spec):
...
```

- Will NOT work in 2.x:

```
my_spec = tf.DeviceSpec.from_string("/CPU:0")
my_spec.parse_from_string("/GPU:0") # <== Will not update my_spec
with tf.device(my_spec):
...
```

Recommended:

Will work in 1.x and 2.x (though deprecated in 2.x):

Will NOT work in 2.x:

In general, `DeviceSpec.from_string` should completely replace `DeviceSpec.parse_from_string`, and `DeviceSpec.replace` should completely replace setting attributes directly.

replace

[View source](#)

Convenience method for making a new `DeviceSpec` by overriding fields.

For instance:

```
## to_string
```

[View source](#)

Return a string representation of this DeviceSpec.

`__eq__`

[View source](#)

Checks if the other DeviceSpec is same as the current instance, eg have same value for all the internal fields.